DATA 260 Homework 6

Farzaan Dalwai

https://github.com/farzaandalwai/data260-0686

SID4: 0686 | PORT_BASE: 8686 | PREFIX: s0686

SEED: 686 | VERIFY_SEED: 260686 | DOMAIN_ID: 6 - Rental Housing Listings

Hardware: Apple M4, 10 cores, 24 GB RAM

Local model: llama3.2:latest | Embedding: all-MiniLM-L6-v2, 384 dimensions

Tested implementation commit: 8130bc6f753f60ba2dbba6c3d481c8fc6caa7a50

Report base commit: 01c4210059c087b8c3bec577f5dc38c420313407 | Proposed tag: hw6 (not created yet)

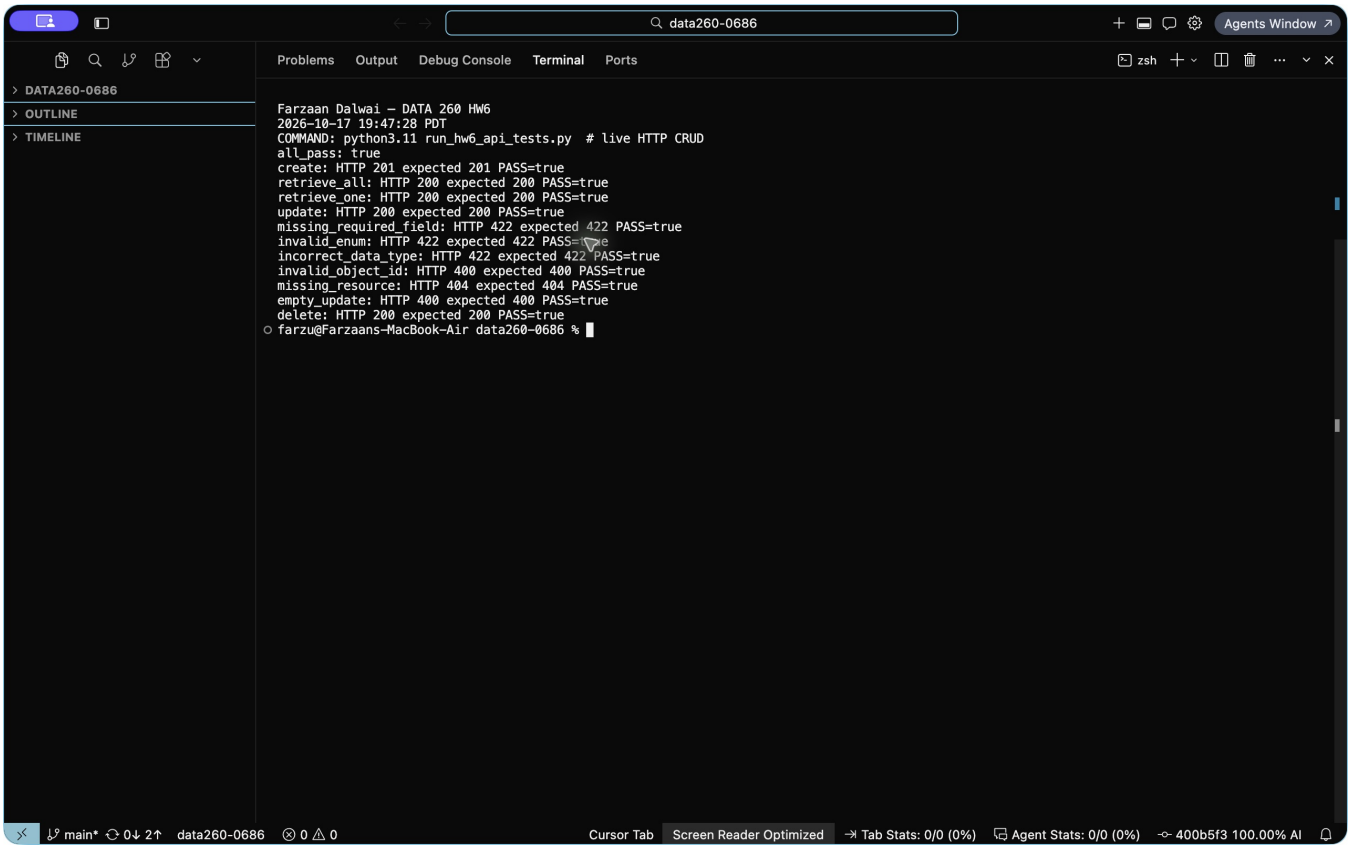
Repository link works and the repository is public. A read-only check found only the owner and no pending invitations. Access for Sbnikitha and supriyaselvanganesan is not confirmed.

Part 1 - MongoDB Document API

I used database s0686_doc and collection rental_documents. Each document has a required title, optional description, status, property type, available date, and automatic created/updated timestamps. Pydantic and MongoDB both enforce required fields, length limits, enums, and dates.

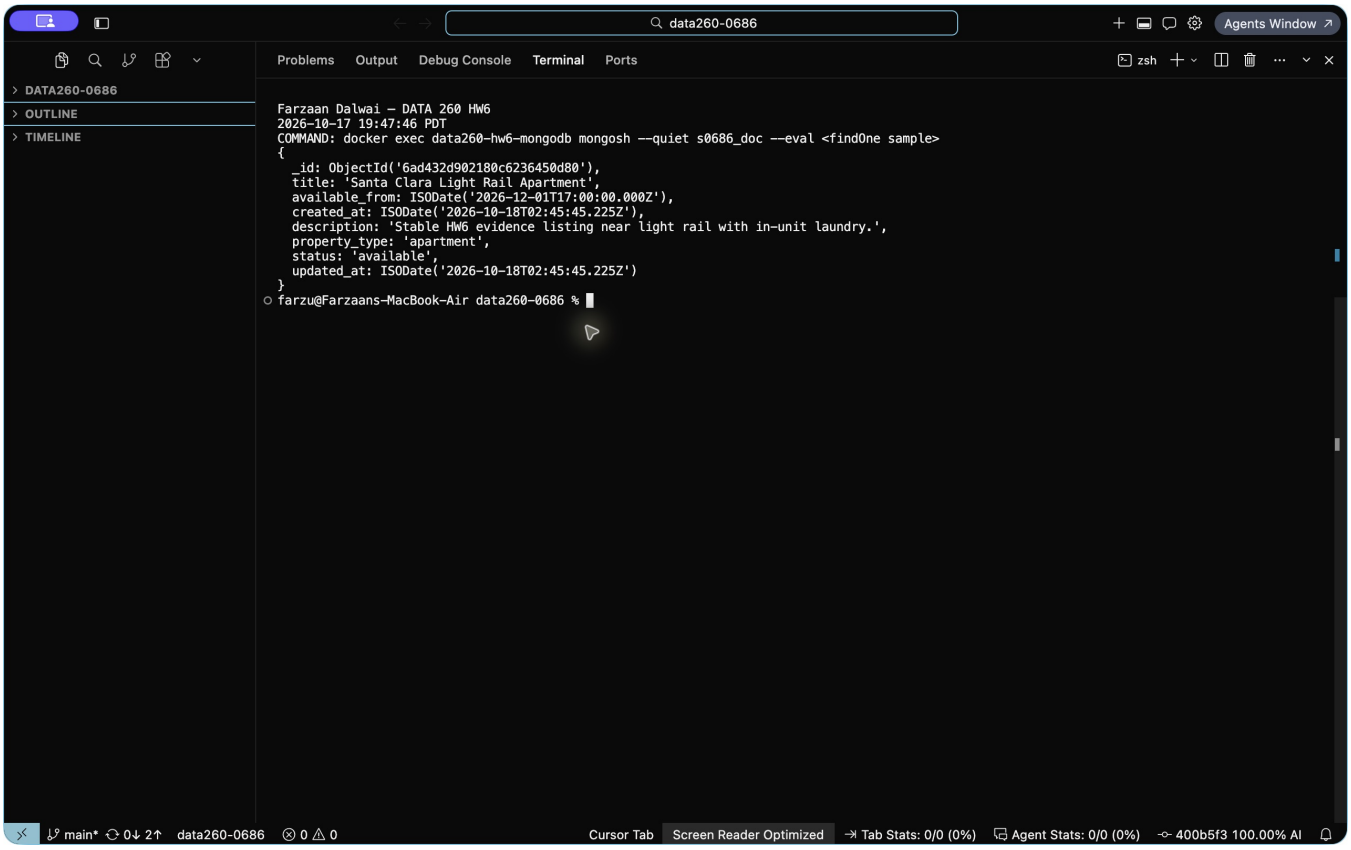
The FastAPI routes support create, list, get by ID, update, and delete. Errors return 400 for bad IDs or empty updates, 404 for missing records, 422 for invalid request data, and a safe 500 response for database errors. CORS and JSON request parsing are enabled in the shared application.

```
@router.post('', status_code=201)
@router.get('')
@router.get('/{listing_id}')
@router.put('/{listing_id}')
@router.delete('/{listing_id}')
# RentalDocumentCreate validates title, enums, and datetime.
```



CRUD results

```
'$jsonSchema': {
  'required': ['title', 'status', 'property_type', 'available_from',
               'created_at', 'updated_at'],
  'properties': {'title': {'maxLength': 100},
                 'status': {'enum': ['available','pending','rented']}}
}
```

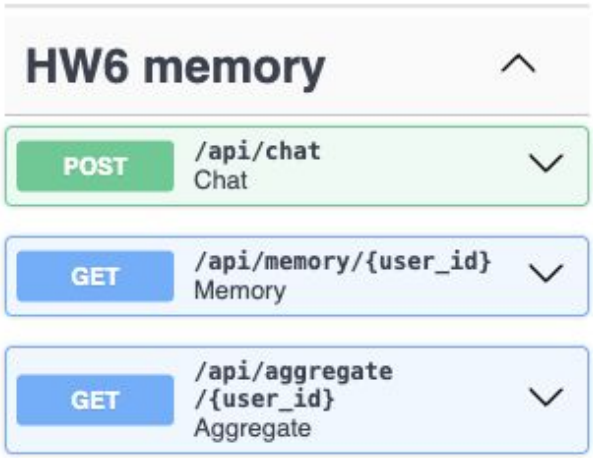


MongoDB document

Part 2 - Short, Long, and Episodic Memory

The same database also has messages, summaries, and episodes. Messages store the conversation. Summaries store session or lifetime text. Episodes store short facts, importance from 0 to 1, and a numeric embedding.

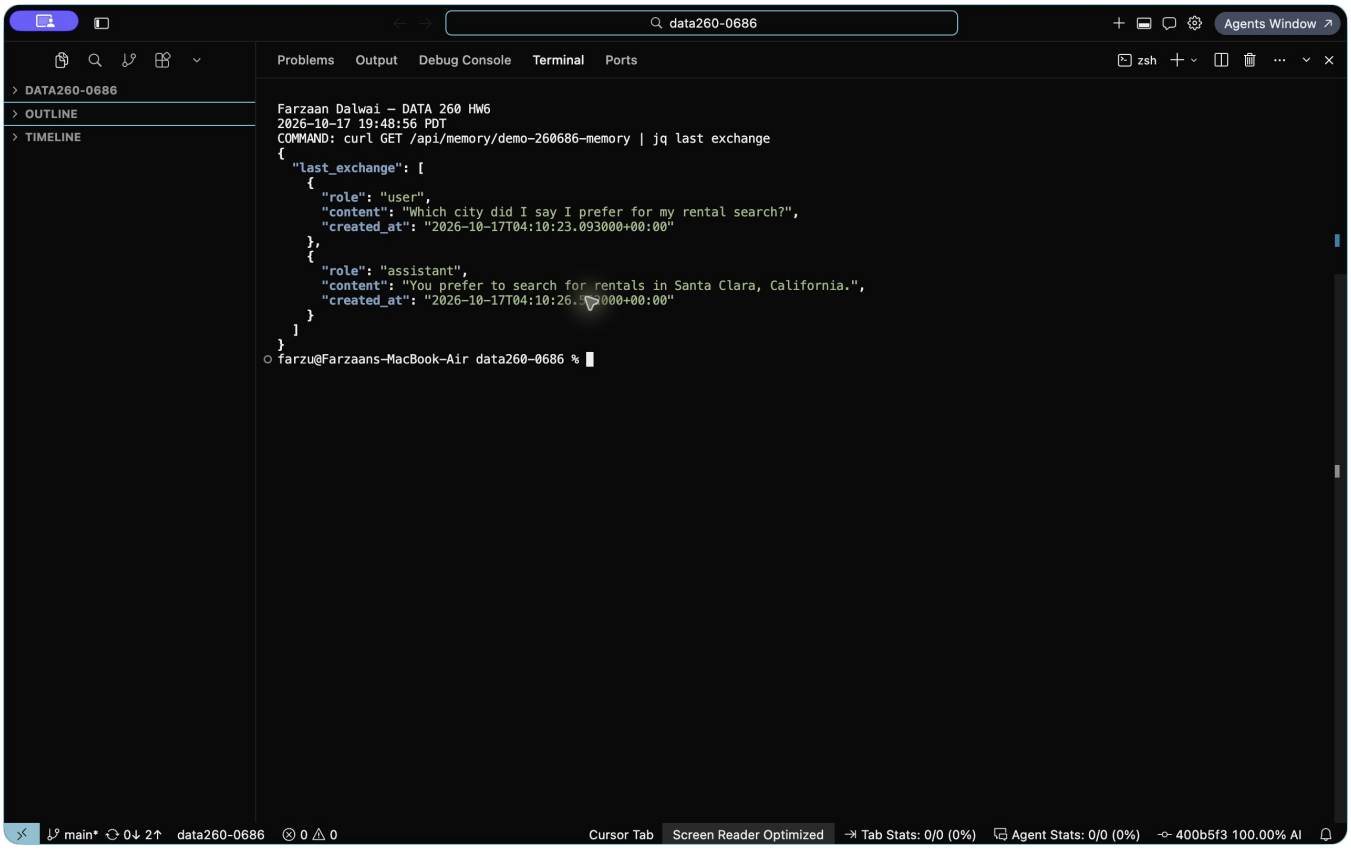
```
@router.post('/api/chat')
@router.get('/api/memory/{user_id}')
@router.get('/api/aggregate/{user_id}')
```



API endpoints

POST /api/chat saves the user message, loads the last N messages, loads session and lifetime summaries, retrieves the most relevant episodes, builds the prompt, calls local Ollama, saves the reply, and updates summaries every five user messages. The response includes the reply and the memory used.

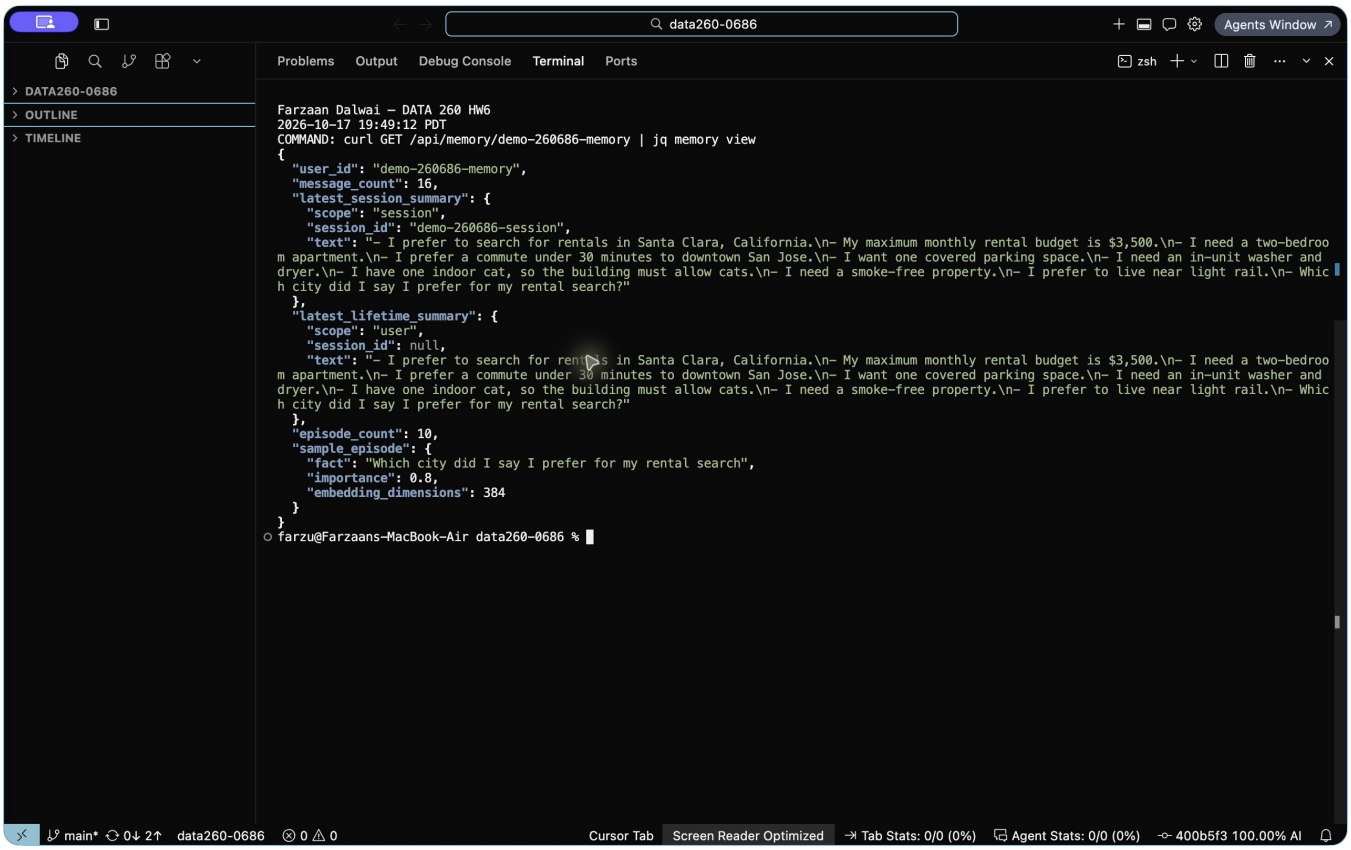
```
context = assemble_prompt(system_primer, summaries, short_window, episodes, message)
reply, usage = model.generate(context)
save_message(user_id, session_id, 'assistant', reply)
return {'reply': reply, 'memory_used': memory_used, 'usage': usage}
```



Chat reply

GET /api/memory/{user_id} returns the last 16 messages, newest session summary, newest lifetime summary, and the last 20 episodes.

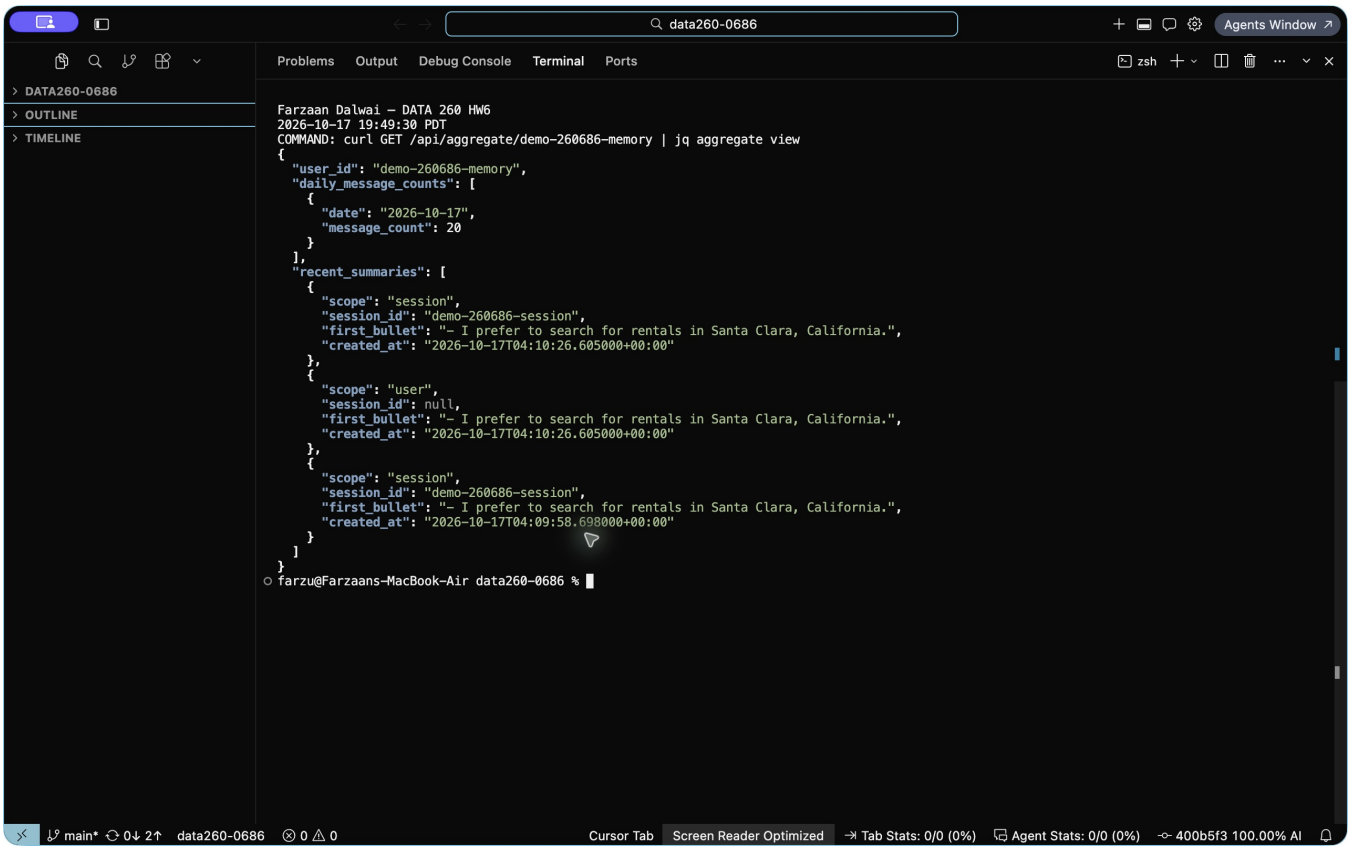
```
return {'messages': last_16_messages(user_id),
        'latest_session_summary': latest_summary(user_id, 'session'),
        'latest_lifetime_summary': latest_summary(user_id, 'user'),
        'episodes': last_20_episodes(user_id)}
```



Memory result

GET /api/aggregate/{user_id} groups messages by date and returns recent session and lifetime summaries.

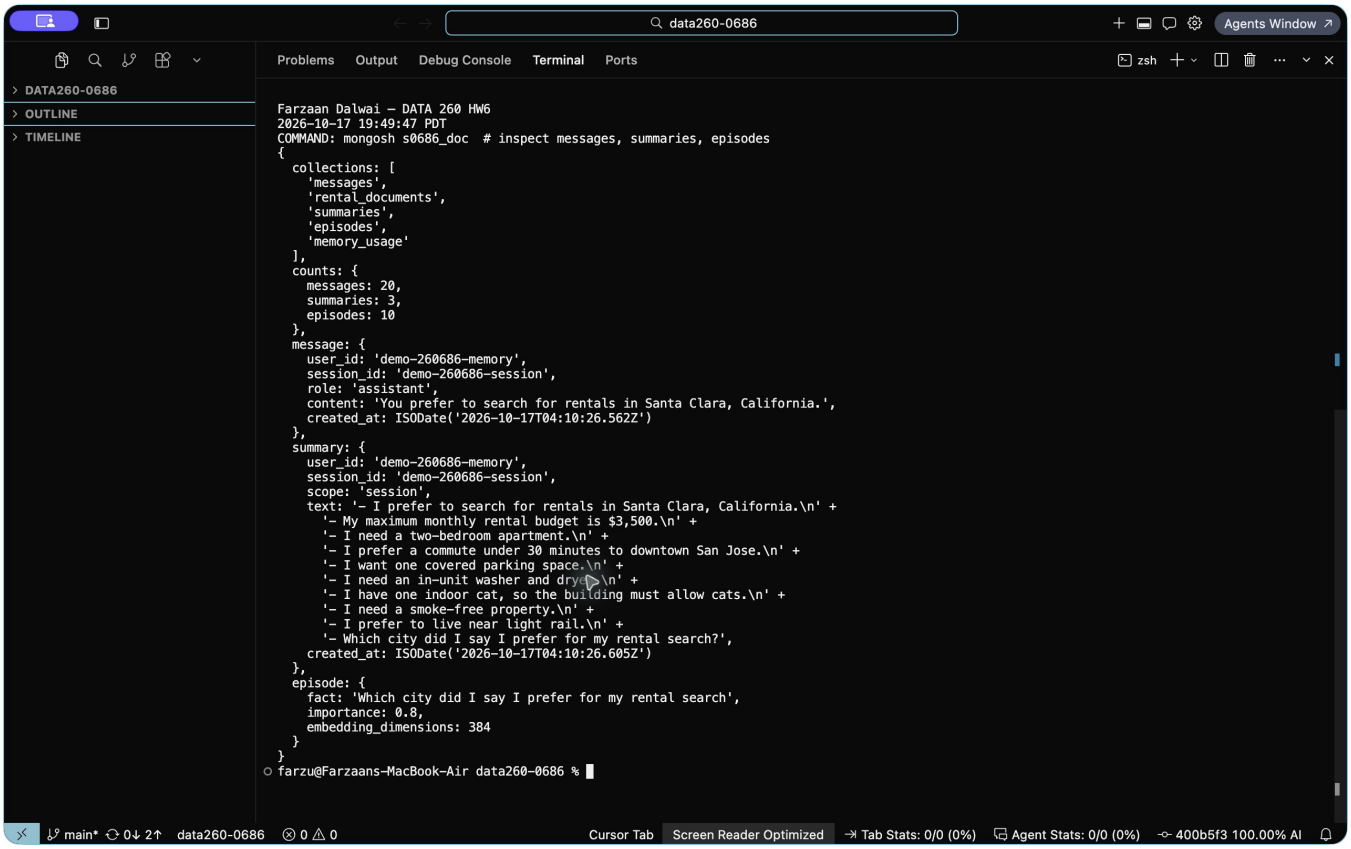
```
pipeline = [
  {'$match': {'user_id': user_id}},
  {'$group': {'_id': {'$dateToString': {'format': '%Y-%m-%d',
      'date': '$created_at'}}, 'message_count': {'$sum': 1}}}
]
```



Aggregate result

I sent 10 messages in one session. SHORT_TERM_N is 12. A summary is made every five user messages. Up to three facts are extracted from each user message and stored with MiniLM embeddings. Cosine similarity returns the top three relevant facts. The test produced 20 messages, 3 summaries, and 10 episodes.

```
messages: user_id, session_id, role, content, created_at
summaries: user_id, session_id, scope, text, created_at
episodes: user_id, session_id, fact, importance, embedding, created_at
```



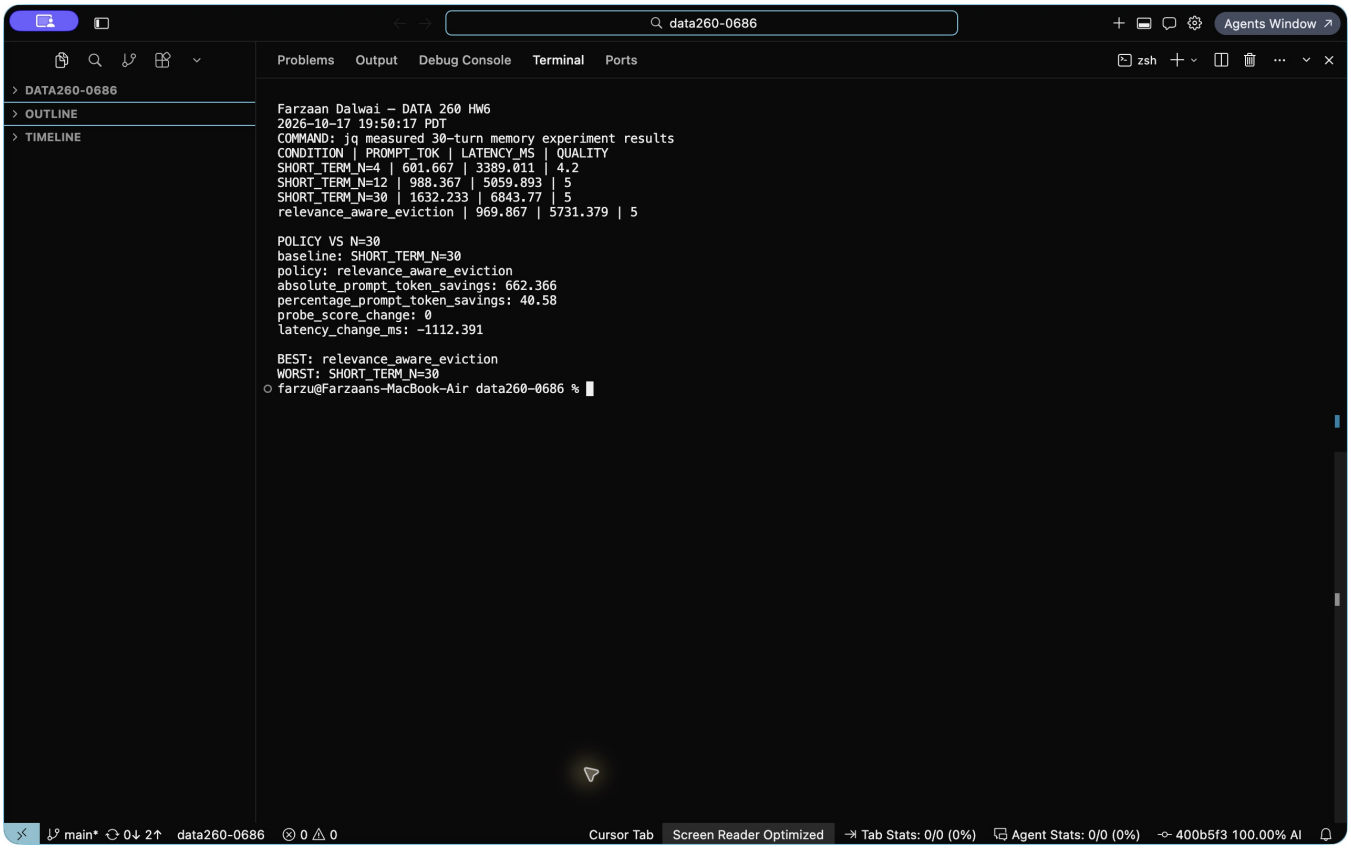
Memory collections

Part 3 - What Memory Costs

Each chat request logs prompt/completion tokens, estimated tokens for the system primer, long-term summary, short-term window and episodes, plus latency, condition, IDs, timestamp, and model settings. The raw data is JSON.

memory_probes.yaml contains five fixed questions, expected answer criteria, and a 1-5 scoring rubric. I ran the same 30-message conversation and the same five probes in a fresh session for each condition.

```
for condition in ['N=4', 'N=12', 'N=30', 'relevance_aware_eviction']:
    run_same_30_messages_in_fresh_session()
    score_same_five_probes()
```



Experiment results

Memory results

Setting	Prompt/turn	Latency ms	Quality
SHORT_TERM_N=4	601.667	3389.011	4.200
SHORT_TERM_N=12	988.367	5059.893	5.000
SHORT_TERM_N=30	1632.233	6843.770	5.000
Eviction policy	969.867	5731.379	5.000

My policy keeps the four newest messages and the eight most relevant older messages. Compared with N=30, it saved 662.366 prompt tokens per turn (40.580%), changed quality by +0.000, and changed latency by -1112.391 ms.

10,000 conversation cost

Setting	Input	Output	Total
SHORT_TERM_N=4	$180.5001	$144.0000	$ 324.5001
SHORT_TERM_N=12	$296.5101	$144.0000	$ 440.5101
SHORT_TERM_N=30	$489.6699	$144.0000	$ 633.6699
Eviction policy	$290.9601	$144.0000	$ 434.9601

Best setting: relevance-aware eviction because it kept quality 5.0 at the lowest cost in the top quality range ($434.9601). Worst setting: SHORT_TERM_N=30 because it had the highest cost in that range ($633.6699). The first sweep was rejected because Ollama silently truncated context. I set num_ctx=16384 and num_predict=96 and reran all conditions.

Part 4 - File Memory and Consolidation

AGENT.md stores project instructions. MEMORY.md stores durable facts and the last consolidation time. The code loads both files and adds them to the system prompt. Consolidation is required above 40

fact lines or after 24 hours.

```
def file_memory_prompt():
    return load_agent_memory() + '\n' + load_fact_memory()

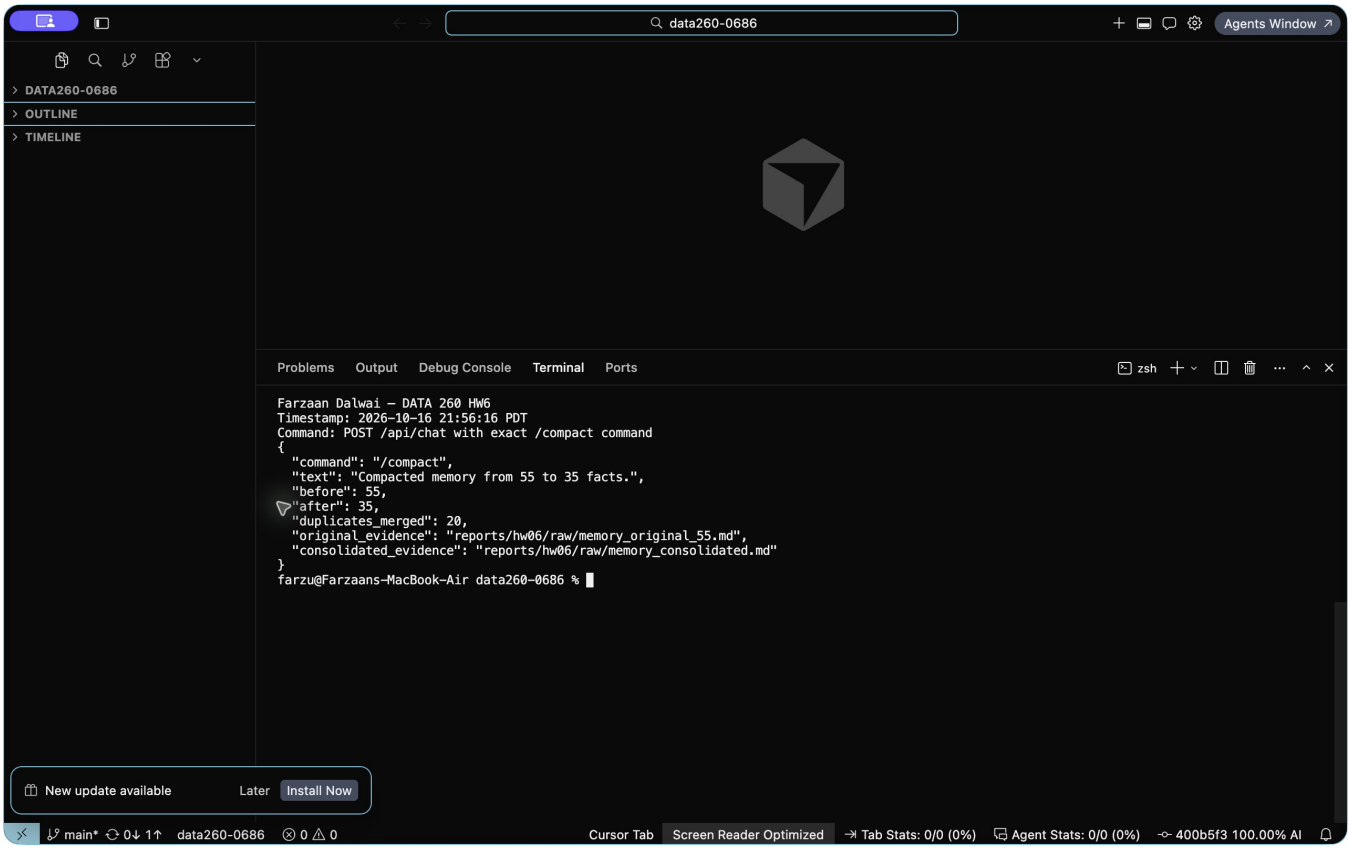
def consolidation_required(now):
    return count_memory_lines() > 40 or hours_since_compact(now) > 24

COMMANDS = {'/memory', '/compact', '/dream', '/help'}
```

/memory displays memory. /compact merges duplicates and resolves conflicts. /dream adds up to three durable facts from recent conversation. /help explains the commands. Callers cannot provide file paths, so only the two root memory files can be accessed.

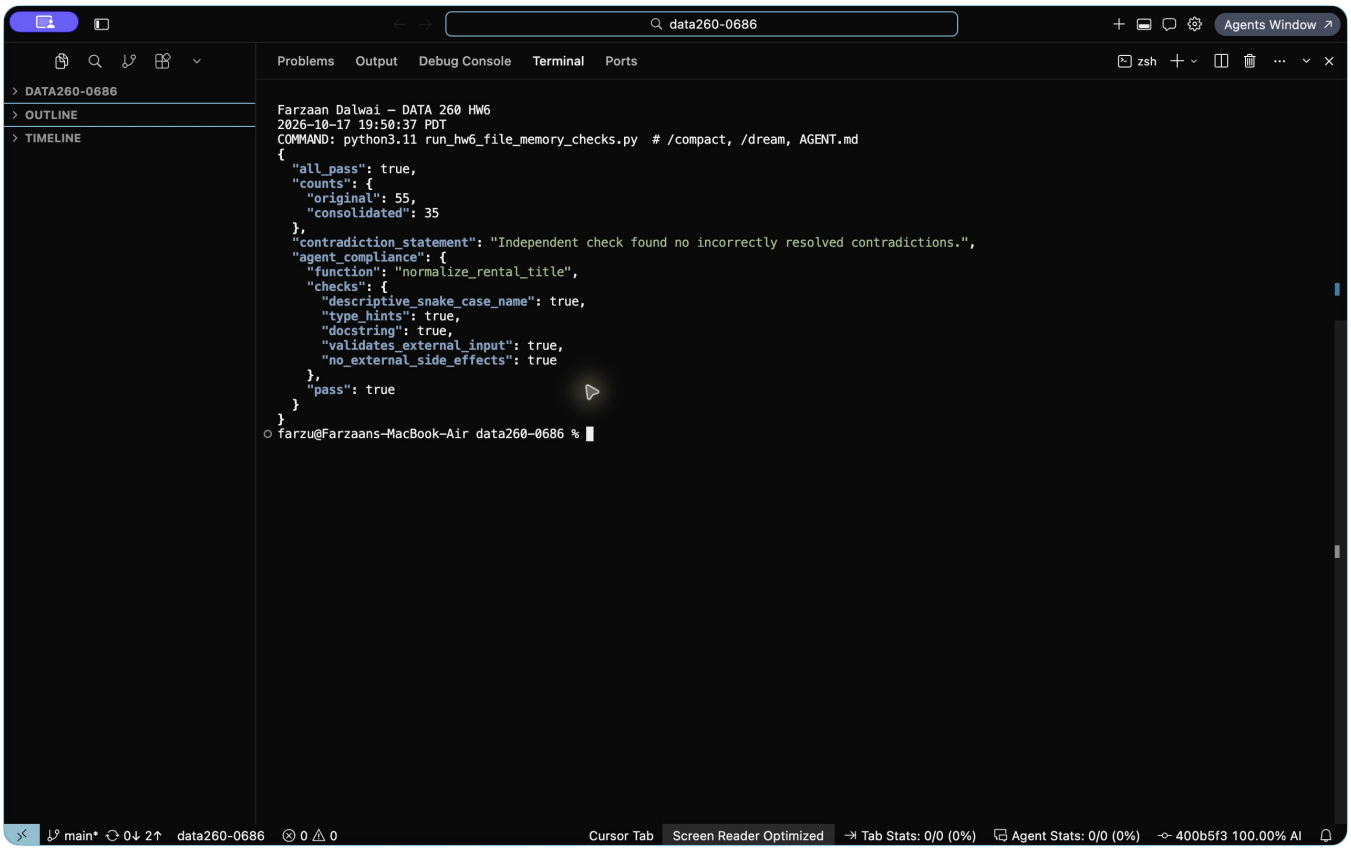
AGENT.md has nine instructions covering the project, stack, coding conventions, testing, and prohibited actions. The normalize_rental_title function passed checks for naming, type hints, a docstring, input validation, and no external side effects.

```
if command == '/compact':
    result = compact_memory()
    return {'before': before, 'after': after, **result}
# Conflicts use confidence, then newest observation; alternatives are retained.
```



Compact result

```
checks = {'descriptive_name': True, 'type_hints': True,
          'docstring': True, 'validates_input': True,
          'no_external_side_effects': True}
assert all(checks.values())
```



Agent verification

The original MEMORY.md had 55 fact lines with duplicates and contradictions (commit 177192e). The consolidated version has 35 lines (commit 9a7c2bd). It merged 20 duplicates. Conflicts were kept with the selected result and reason, and the independent check found no incorrectly resolved contradiction.

Memory comparison

Inspectability: MongoDB is easy to query; Markdown is easiest to read and review in Git.

Restart persistence: MongoDB survives in its volume; Markdown survives in files and commits.

Many sessions: MongoDB scales better with indexed user/session queries; one Markdown file becomes noisy.

Must-not-forget facts: MongoDB is good for scored user facts; Markdown is best for a few critical rules.

Choice: use MongoDB for conversation memory and retrieval, and file memory for small, auditable project rules.

AI Use

1. I used Codex for repository inspection, implementation, debugging, testing, experiments, evidence capture, and report drafting. I supplied the assignment, repository, configuration, and local services.

2. The first N=30 experiment was unsuitable because the prompt count stopped near 2,050 tokens.

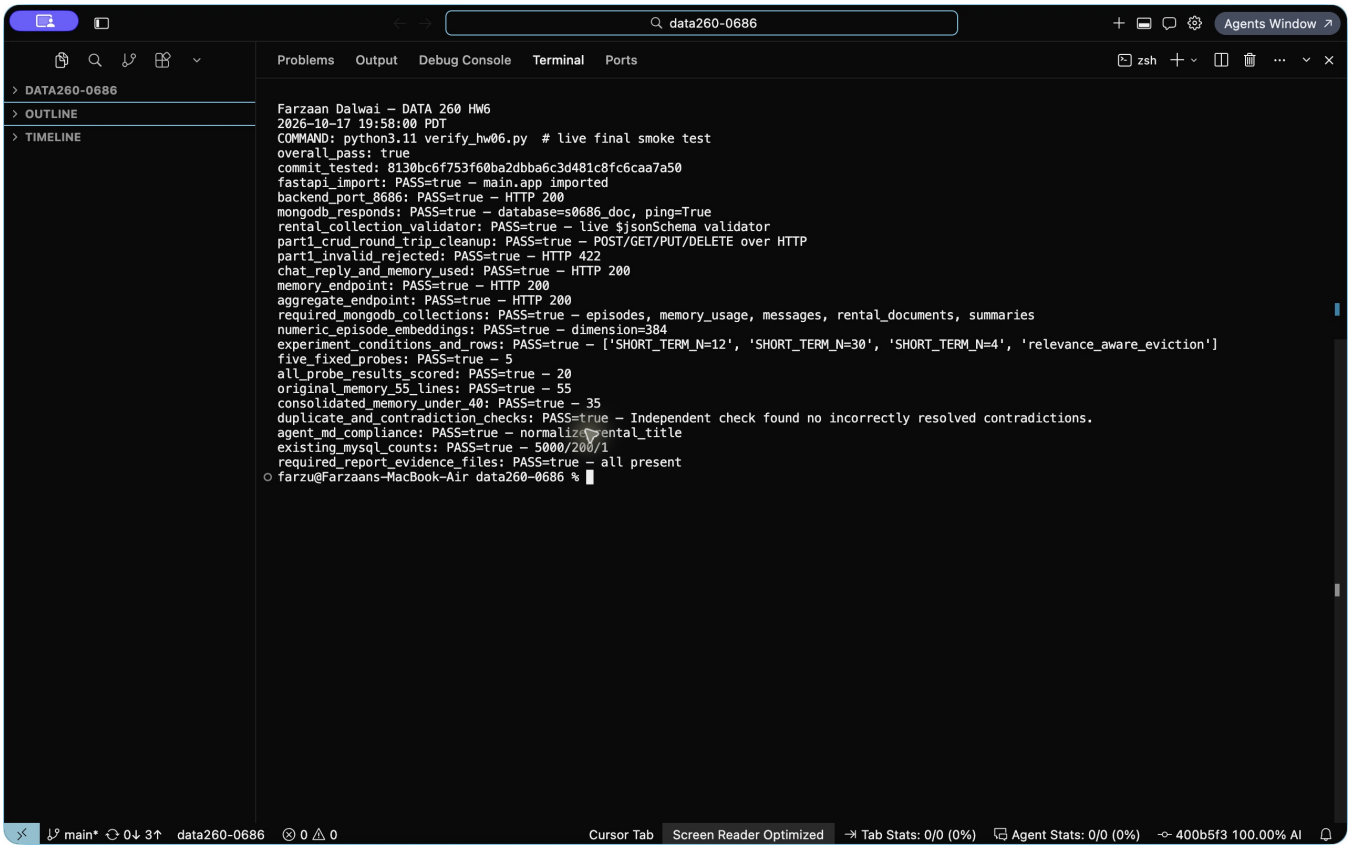
3. I found the problem by comparing exact Ollama prompt_eval_count values across conditions.

4. I preserved the bad run, set num_ctx=16384 and num_predict=96, and reran every condition in fresh sessions. The corrected N=30 prompt counts exceeded the old limit.

Verification

The self-check tested FastAPI, MongoDB, CRUD and validation, local chat, memory, aggregate output, collections, 384-dimensional embeddings, experiment files, file memory, AGENT.md, and unchanged MySQL counts.

```
python3.11 verify_hw06.py
# Live smoke test writes reports/hw06/verification.json
```



Final verification

Result: 20/20 checks passed. overall_pass=true.

The report and evidence are complete locally. Push, tag creation, and collaborator access changes were not performed.